Notes for C++

Section 2 : the if statement

The if statement allows the program to carry out different instructions in different situations. Here is an example program that uses an if statement:

```
#include <iostream>
using namespace std;

int value;

int main() {
  cout << "Enter an integer value ";
  cin >> value;
  if (value < 10) {
    cout << "This value is less than 10";
  }
}
```

If we run this program and enter 3, the program will display
`This value is smaller than 10`

But what happens if we run the program and enter 15? Nothing is displayed.

The syntax of the if statement (or how to write the if statement)

- it starts with the reserved word if
- there is a ***condition*** (or a test) inside ordinary brackets (parentheses)
- there are statements inside the ***body*** of the if statement – inside the curly brackets

The semantics of the if statement (or how does it work?)

If the condition is true, then the statements in the body are executed. Otherwise the statements in the body are ignored.

Conditions

Every if statement has a ***condition*** (a test inside the brackets).
The result of a test is always `true` or `false`.
You need to know the conditions that are available in a C++ program. They include:

`(a < 9)`	is `a` less than 9?
`(a > 2)`	is `a` greater than 2?
`(a == 4)`	is `a` equal to 4?
`(a <= 7)`	is `a` less than or equal to 7?
`(a >= 3)`	is `a` greater than or equal to 3?
`(a != 5)`	is `a` not equal to 5?

Several if statements in a program

A program can have several if statements. For example:

```
#include <iostream>
using namespace std;

int value;

int main() {
  cout << "Enter an integer value ";
  cin >> value;
  if (value < 10) {
    cout << "This value is less than 10";
  }
  if (value > 10) {
    cout << "This value is greater than 10";
  }
}
```

This program has two couts but only two of them will be executed. Or maybe only one…?
What does it display on the screen?
What happens if we run the program and enter 15?
What happens if we run the program and enter 10?

People don't like programs that sometimes have a blank display. Always try to avoid this. We can solve this problem in this program by having three if statements:

```
#include <iostream>
using namespace std;

int value;

int main() {
  cout << "Enter an integer value ";
  cin >> value;
  if (value < 10) {
    cout << "This value is less than 10";
  }
  if (value > 10) {
    cout << "This value is greater than 10";
  }
  if (value == 10) {
    cout << "This value is equal to 10";
  }
}
```

Logical Operators

We can link two (or more) tests together to make one multiple-condition test. This is done with **AND** and **OR**. The symbol for **AND** is && (two ampersands) and the symbol for **OR** is || (two bars).

The result of the final test is found by looking at the ***truth tables*** for **AND** and **OR**.

Test 1	Test 2	Result of AND
True	True	True
True	False	False
False	True	False
False	False	False

Test 1	Test 2	Result of OR
True	True	True
True	False	True
False	True	True
False	False	False

Here is an example program:

```
#include <iostream>
using namespace std;

int value;

int main() {
  cout << "Enter an integer value ";
  cin >> value;
  if ((value >= 1) && (value <= 6)) {
    cout << "This is a valid dice throw\n";
  }
  if ((value == 27) || (value == 83)) {
    cout << "This value is either 27 or 83\n";
  }
}
```

Things to note about this program:

- each test is inside brackets and the full combined test is also inside brackets
- the symbol for **AND** is && and the symbol for **OR** is `||`
- use **AND** to check for a ***range of values*** (in this case, from 1 to 6)

Test this piece of program out. What happens? Change to other values and test again.

The third logical operator is called **NOT** (the symbol is !). **NOT** reverses the value of a test.
For example: `( ! (value == 9))` is another way of writing `(value != 9)`.

Priority of Operators

In arithmetic we use the BEDMAS rule to remember the ***priority*** of the operators.

For example: `(2 + 3) * 4` produces a different result from `2 + 3 * 4`.

In logic the rule is **NOTANDOR**. Thus NOT is the most important and AND is more important than OR.

For example, adding a set of brackets changes the result of this condition:

```
  if ((m <= 8) || (m > 15) && (m == 3)) {
```

Now include an extra set of brackets like this:

```
  if ( ( (m <= 8) || (m > 15) ) && (m == 3)) {
```

Test these conditions with m = 8 and you get different results.

Just like with BEDMAS, brackets can override NOTANDOR.

It is important to practice! Here is an exercise to practice. Predict what the output will be before writing and testing the program. Note the position of the brackets.

```
  m = 24;
  if ( (m != 32) && (m != 24) ) {
    cout << "Result one\n";
  }
  if ( ! (m != 32) && (m != 24) ) {
    cout << "Result two\n";
  }
  if ( ! ( (m != 32) && (m != 24) ) ) {
    cout << "Result three\n";
  }
```

Change the value of m from 24 to some other value. Keep practicing!

Nested if statements

The body of an if statement can contain any number of legal C++ statements. An if statement is also a legal C++ statement and can thus appear inside the body of an if statement. When we have one if statement inside another one, we say that we have "nested" if statements. You will see examples of nested if statements as we work our way through the course. Here is an example:

```
#include <iostream>
using namespace std;

int value;

int main() {
  cout << "Enter an integer value ";
  cin >> value;
  if (value < 10) {
    cout << "This value is less than 10 ";
    if (value >= 0) {
      cout << "and is positive.\n";
    }
    if (value < 0) {
      cout << "and is negative.\n";
    }
  }
  if (value >= 10) {
    cout << "This value is greater than or equal to 10\n";
  }
}
```

Note: there could be more than one if statement inside an if statement
Could there be an if inside an if that is inside an if?

The if statement with an else

Previously we wrote a program that sometimes displayed nothing on the screen. We corrected it by adding an extra if statement to cover the case with no output. We could have corrected it by using an `else` with the if statement, like this:

```
#include <iostream>
using namespace std;

int value;

int main() {
  cout << "Enter an integer value ";
  cin >> value;
  if (value < 10) {
    cout << "The value is less than 10";
  } else {
    cout << "The value is greater than or equal to 10";
  }
}
```

This program will ***either*** display `The value is less than 10` ***or*** it will display `The value is greater than or equal to 10`. Note that it will ***never display both***.

The syntax of the if statement with else (or how to write an if with else)

- the if part you already know
- the else part has ***no condition*** (no test)
- there are statements inside the ***body*** of the else part – inside the curly brackets

The semantics of the if statement with else (or how does it work?)

- if the test is true then the statements in the ***body of the if*** are executed.
- if the test is false then the statements in the ***body of the else*** are executed.
- one part of an if/else statement will **always** run, unlike ifs alone, which may or may not run
- an if statement does not have to include else, it is optional

The if statement with else if and else – this example has several versions

Using if with else sorts out situations where there are two mutually exclusive situations. What if there are more than two? We use **if with else if** as many times as necessary, with a final else (which is optional – you could have all else ifs).

Some universities calculate the class of pass based on the final result for a course. (Massey university does not do this). Write a program to display the class of pass.

Version 1 of the program:

```
#include <iostream>
using namespace std;

int result;

int main() {
  cout << "Enter the final result ";
  cin >> result;
  if (result >= 75) {
    cout << "First class pass";
  } else if (result >= 67) {
```

```
    cout << "Upper second class pass";
  } else if (result >= 60) {
    cout << "Lower second class pass";
  } else if (result >= 50) {
    cout << "Third class pass";
  } else {
    cout << "Fail";
  }
}
```

The syntax of the if statement with else if (or how to write the if with else if)

- `else if` is used for the different alternatives and each else if part has a condition
- the use of else for the final alternative is optional (could be if with only else ifs)
- where else is used, its format is as already seen - no condition

The semantics of the if statement with else if (or how does it work?)

- in the same way as one part of an if/else statement will **always** run, one (and only one) part of an if with any number of else ifs and else will **always** run

Another interesting thing to notice in this program:

- **the order in this particular problem is important** – it won't work if you change the order.

Any program can be written in a number of different ways. If you don't like using else if then we can rewrite this program in a different way. Note that this is a separate, new program.

Version 2 of the program:

```
#include <iostream>
using namespace std;

int result;

int main() {
  cout << "Enter the final result ";
  cin >> result;
  if (result >= 75) {
    cout << "First class pass";
  }
  if (result >= 67) {
    cout << "Upper second class pass";
  }
  if (result >= 60) {
    cout << "Lower second class pass";
  }
  if (result >= 50) {
    cout << "Third class pass";
  } else {
    cout << "Fail";
  }
}
```

Oops, version 2 of the program doesn't work. How do we know it doesn't work? You cannot simply remove else if and expect the same result. In fact it sometimes works (e.g. if the result is 38) and sometimes does not work (e.g. if the result is 84). What causes the problem?

Let us write another version of the program to fix the problem.

Version 3 of the program:

```
#include <iostream>
using namespace std;

int result;

int main() {
  cout << "Enter the final result ";
  cin >> result;
  if (result >= 75) {
    cout << "First class pass";
  }
  if ((result >= 67) && (result < 75)) {
    cout << "Upper second class pass";
  }
  if ((result >= 60) && (result < 67)) {
    cout << "Lower second class pass";
  }
  if ((result >= 50) && (result < 60)) {
    cout << "Third class pass";
  }
  if (result < 50) {
    cout << "Fail";
  }
}
```

Interesting things to notice in version 3 of the program:

- it does not use `else if`
- the order is not important – why not?
- does it work for every result? What if the result is 278?

Always test your programs

Run the program several times with different data. Ensure that you test the program for all the ***different situations*** that may arise. For example, this program should be tested for results in every class. The worst problem is a program that works in some situations but not in others (for an example of a bad program, see version 2 of the program).

Example: A program to find the biggest (maximum) number

Take any three numbers, for example 19 27 2. Which number is the maximum? You probably got it right but how did you know what to do? What algorithm did you use to calculate the biggest number? What if there were 10,000 number and not just 3 numbers?

```
#include <iostream>
using namespace std;

int a, b, c, big;

int main() {
  cout << "Enter three integer values\n";
  cin >> a >> b >> c;
  big = a;
  if (big < b) {
    big = b;
  }
  if (big < c) {
    big = c;
  }
  cout << "The biggest number is " << big;
}
```

Interesting things to notice in this program:

- we assume that `a` is the biggest number and then we check against the others
- we keep changing the variable called `big`. Remember this useful trick!
- make an example and use the example to help you design the program

Example: A program to calculate the minimum number of coins

Assume that you owe someone an amount that is less than $1. What is the minimum number of coins that you need to pay them? Example: 90 cents = 50cents + 20cents + 20cents so you need 3 coins. Assume that the amount is in cents, it must be less than one dollar and it must be a multiple of 10.

```
#include <iostream>
using namespace std;

/* calculates minimum number of coins needed to pay an amount.
   The amount must be less than $1 and must be a multiple of 10. */

int cents, total;

int main() {
  cout << "Enter an amount in cents ";
  cin >> cents;
  if (cents >= 100) {
    cout << cents << " is greater than one dollar\n";
  } else if (cents % 10 != 0) {
    cout << cents << " is not a multiple of 10\n";
  } else {
    cout << cents << " cents = ";
    total = 0;
    if (cents >= 50) {
       cents = cents - 50;
       cout << "50c + ";
       total += 1;
    }
```

```
    if (cents >= 20) {
       cents = cents - 20;
       cout << "20c + ";
       total += 1;
    }
    if (cents >= 20) {
       cents = cents - 20;
       cout << "20c + ";
       total += 1;
    }
    if (cents >= 10) {
       cents = cents - 10;
       cout << "10c";
       total += 1;
    }
    cout << " You need " << total << " coins to pay.";
  }
}
```

Interesting things to notice in this program:

- note the `else if` notation
- the order is important – why?
- make an example and use the example to help you design the program
- some of the statements in the else are if statements (ifs nested inside else)
- this program does not display the results well – what is the problem? Can you fix it?

Calculating leap year

Write a program to calculate if a year is a leap year or not. There are two rules:

1) A year is a leap year if it can be divided exactly by 4 but ***not*** exactly by 100.
   For example, 1900 was not a leap year.

2) A year that divides exactly by 400 is a leap year. For example, 2000 was a leap year.

For more information on leap years see:
https://en.wikipedia.org/wiki/Leap_year

Here is a program that calculates if a year is a leap year. Technically, there is a minimum year and if you enter a year before that, the output won't make sense, e.g. do not enter year 1. Let's not worry too much about that and enter reasonable years.

```
#include <iostream>
using namespace std;

int year, leapyear;

int main() {
  cout << "Enter a year ";
  cin >> year;
  if (year % 4 == 0) {
    if (year % 100 == 0) {
      if (year % 400 == 0) {
        leapyear = 1;
```

```
      } else {
        leapyear = 0;
      }
    } else {
      leapyear = 1;
    }
  } else {
    leapyear = 0;
  }
  if (leapyear == 1) {
    cout << year << " is a leap year.";
  } else {
    cout << year << " is NOT a leap year.";
  }
}
```

Interesting things to notice in this program:

- the remainder operation is used to check if a year is divided exactly by 4 or 100 or 400
- the leapyear variable is used to keep track of whether a year is a leap year or not
- the ***nested if statements*** (in other words: one if statement inside another one)

Any program can be written in a number of different ways. Some people find all those else's confusing. Let us rewrite the program in a different way. Note that this is a separate, new program.

```
#include <iostream>
using namespace std;

int year, leapyear;

int main() {
  cout << "Enter a year ";
  cin >> year;
  leapyear = 0;
  if (year % 4 == 0) {
    leapyear = 1;
  }
  if (year % 100 == 0) {
    leapyear = 0;
  }
  if (year % 400 == 0) {
    leapyear = 1;
  }
  if (leapyear == 1) {
    cout << year << " is a leap year.";
  } else {
    cout << year << " is NOT a leap year.";
  }
}
```

Interesting things to notice in this program:

- the word else is not used (does this make the program easier to read?)
- we start the program by making an **assumption** – we set leapyear to 0 which means we are assuming that the year is not a leap year. We may change the leapyear variable later on.

- the order of the first three if statements is very important…. Why?

Testing

You must ***test your programs***. Run the program several times with different data. Ensure that you test the program for all the ***different situations*** that may arise. For example, this program should be tested **(a)** for a year that does not divide by 4 (e.g. 1999), **(b)** for a year that does divide by 4 (e.g. 2012), **(c)** for a year that divides by 100 (e.g. 1900) and finally **(d)** for a year that divides by 400 (e.g. 2000).